Unit 4 - Lesson 1D Reference Guide

References vs Copies: THE GOTCHA!

🔄 Mutable vs Immutable Review

Immutable Types (SAFE - Cannot be changed)

Type	Example	What Happens
`int`	`x = 5`	Operations create NEW values
`float`	`x = 3.14`	Original never modified
`str`	`x = "hello"`	Strings are immutable
`bool`	`x = True`	Cannot change in place

```
# Immutable example - SAFE!
def add_ten(num):
    num = num + 10  # Creates NEW integer
    return num

score = 100
new_score = add_ten(score)
print(score)      # 100 - Original unchanged! ✅
print(new_score)  # 110
```

Mutable Types (DANGER - Can be changed in place!)

Type	Example	What Happens
`list`	`x = [1, 2, 3]`	Can be modified in place!
`dict`	`x = {"a": 1}`	Can be modified in place!

```python
# Mutable example - DANGER!
def add_item(items):
    items.append("new")  # Modifies the SAME list!
    return items

original = ["a", "b", "c"]
result = add_item(original)
print(original)  # ['a', 'b', 'c', 'new'] - CHANGED! 😱
```

❌ The Problem: References

What DOESN'T Work

```python
original = ["sword", "shield", "potion"]
backup = original  # ❌ This is NOT a copy!

original.append("tnt")

print(original)  # ['sword', 'shield', 'potion', 'tnt']
print(backup)    # ['sword', 'shield', 'potion', 'tnt'] 😱
```

Why It Fails

```
original ──────┐
               ├──▶ ["sword", "shield", "potion", "tnt"]
backup   ──────┘

TWO names, ONE list!
```

⚠️ **The = operator with lists creates a REFERENCE, not a copy!**

✅ Three Ways to Make REAL Copies

Method 1: .copy() - RECOMMENDED ⭐

```
original = ["sword", "shield", "potion"]
backup = original.copy()  # ✅ Real copy!

original.append("tnt")

print(original)  # ['sword', 'shield', 'potion', 'tnt']
print(backup)    # ['sword', 'shield', 'potion'] ✅ Safe!
```

Why it's best: Crystal clear what you're doing!

Method 2: [:] - The Slice Trick

```
original = ["sword", "shield", "potion"]
backup = original[:]  # ✅ Slice = copy!

original.append("tnt")

print(backup)  # ['sword', 'shield', 'potion'] ✅ Safe!
```

Note: `[:]` means "slice from start to end" = entire list copy

Method 3: list() - The Constructor

```
original = ["sword", "shield", "potion"]
backup = list(original)  # ✅ New list from old!

original.append("tnt")

print(backup)  # ['sword', 'shield', 'potion'] ✅ Safe!
```

📋 Copy Methods Comparison

Method	Syntax	Creates Copy?	Clarity
Assignment	`b = a`	❌ NO - Reference!	-
`.copy()`	`b = a.copy()`	✅ YES	⭐⭐⭐ Best
Slice	`b = a[:]`	✅ YES	⭐⭐ Good
`list()`	`b = list(a)`	✅ YES	⭐⭐ Good

Recommendation: Use `.copy()` for clarity!

🔍 The `is` Operator

What It Does

The `is` operator checks if two variables point to the **exact same object** in memory.

```
a = [1, 2, 3]
b = a           # Reference
c = a.copy()    # Copy

print(b is a)   # True  - Same object!
print(c is a)   # False - Different objects!
```

`is` vs `==`

Operator	Checks	Example
`==`	Same **values**?	`[1,2] == [1,2]` → `True`
`is`	Same **object**?	`[1,2] is [1,2]` → `False`

```python
list_a = [1, 2, 3]
list_b = [1, 2, 3]  # Same values, different object!
list_c = list_a      # Same object!

print(list_a == list_b)  # True  (same values)
print(list_a is list_b)  # False (different objects)

print(list_a == list_c)  # True  (same values)
print(list_a is list_c)  # True  (same object!)
```

🛡️ Writing Safe Functions

❌ UNSAFE Function (Modifies Original)

```python
def add_bonus_item(inventory):
    """DANGER: Modifies the original list!"""
    inventory.append("mystery_box")
    return inventory

# Usage
player_items = ["sword", "shield"]
bonus_inv = add_bonus_item(player_items)

print(player_items)  # ['sword', 'shield', 'mystery_box'] 😱 Changed!
```

✅ SAFE Function (Preserves Original)

```
def add_bonus_item_safe(inventory):
    """SAFE: Works on a copy, original unchanged!"""
    new_inventory = inventory.copy()  # Make a copy first!
    new_inventory.append("mystery_box")
    return new_inventory

# Usage
player_items = ["sword", "shield"]
bonus_inv = add_bonus_item_safe(player_items)

print(player_items)  # ['sword', 'shield'] ✅ Original safe!
print(bonus_inv)     # ['sword', 'shield', 'mystery_box']
```

The Pattern

```
def safe_function(some_list):
    # Step 1: Copy at the start!
    working_copy = some_list.copy()

    # Step 2: Work on the copy
    working_copy.append("something")
    working_copy.sort()
    # ... any modifications ...

    # Step 3: Return the modified copy
    return working_copy
```

🤔 When Do You Need a Copy?

✅ YES - Make a Copy When:

Situation	Example
Making a backup	`backup = original.copy()`
Function shouldn't modify input	`working = param.copy()`

Situation	Example
Sorting for display only	`display = sorted(original)`
Testing/debugging	Keep original for comparison
Experimenting	Try things without risk

❌ NO - Don't Need a Copy When:

Situation	Why
You WANT changes to affect both	Intentional shared reference
Just reading/iterating	Not modifying anything
Function is MEANT to modify	Like `list.sort()`

❌ Common Mistakes

Mistake 1: Forgetting to Copy in Functions

```
# ❌ BAD
def process(data):
    data.sort()  # Oops, modified original!
    return data

# ✅ GOOD
def process(data):
    working = data.copy()
    working.sort()
    return working
```

Mistake 2: Thinking = Makes a Copy

```
# ❌ BAD - Not a copy!
backup = original

# ✅ GOOD - Real copy!
backup = original.copy()
```

Mistake 3: Copying Too Late

```
# ❌ BAD - Damage already done!
def broken(items):
    items.append("x")        # Already modified original!
    backup = items.copy()    # Too late!
    return backup

# ✅ GOOD - Copy first!
def fixed(items):
    backup = items.copy()    # Copy BEFORE any changes!
    backup.append("x")
    return backup
```

Mistake 4: Confusing `is` and `==`

```
a = [1, 2, 3]
b = [1, 2, 3]

# ❌ Wrong understanding
if a is b:  # False! Different objects!
    print("Same")

# ✅ Correct
if a == b:  # True! Same values!
    print("Equal contents")
```

📝 Practice Exercises

Exercise 1: Safe Backup

Write code that creates a safe backup of a playlist.

```
playlist = ["Song A", "Song B", "Song C"]
# Create a backup that won't change when playlist changes
backup = _______________

playlist.append("Song D")
# backup should still be ["Song A", "Song B", "Song C"]
```

Exercise 2: Safe Function

Fix this function to not modify the original:

```
def get_sorted_scores(scores):
    """Return sorted scores without modifying original."""
    scores.sort(reverse=True)  # ❌ This modifies original!
    return scores
```

Exercise 3: Predict the Output

What prints?

```
a = ["x", "y"]
b = a
c = a.copy()
a.append("z")

print(f"a: {a}")
print(f"b: {b}")
print(f"c: {c}")
```

Exercise 4: Check Identity

What does each print?

```python
list1 = [1, 2, 3]
list2 = list1
list3 = list1.copy()

print(list1 is list2)  # ?
print(list1 is list3)  # ?
print(list1 == list3)  # ?
```

✅ Exercise Solutions

Exercise 1: Safe Backup

```python
backup = playlist.copy()
```

Exercise 2: Safe Function

```python
def get_sorted_scores(scores):
    """Return sorted scores without modifying original."""
    working = scores.copy()  # ✅ Copy first!
    working.sort(reverse=True)
    return working

# OR even simpler:
def get_sorted_scores(scores):
    return sorted(scores, reverse=True)  # sorted() returns new list!
```

Exercise 3: Predict the Output

```
a: ['x', 'y', 'z']
b: ['x', 'y', 'z']    # b is reference to a
c: ['x', 'y']         # c is independent copy
```

Exercise 4: Check Identity

```
print(list1 is list2)  # True  - Same object
print(list1 is list3)  # False - Different objects
print(list1 == list3)  # True  - Same values
```

🎯 Quick Reference Card

The Golden Rules

1. `=` **with lists = REFERENCE** (same list, two names)
2. `.copy()` **= REAL COPY** (independent lists)
3. **Functions receive references** to lists
4. **Copy FIRST** in functions if you shouldn't modify original
5. `is` = same object, `==` = same values

Copy Cheat Sheet

```
# ❌ Reference (SAME list)
backup = original

# ✅ Copy (SEPARATE lists)
backup = original.copy()    # Best! ⭐
backup = original[:]        # Also works
backup = list(original)     # Also works
```

Safe Function Template

```
def safe_function(input_list):
    working = input_list.copy()  # Always copy first!
    # ... modify working ...
    return working
```

Unit 4 - Lesson 1D Complete! 🎉

Next up: **Dictionaries - Key-Value Power!** 📚